

Assessment 3 — Must Explain

Measurement provenance. Every number below is from an **actual run on this machine** — Intel Core i7-10700 @ 2.90 GHz, 32 GB RAM, **CPU-only (no GPU)**, Windows 11, Python 3.12, Surya 0.6.13 + BGE-M3 — on 2026-06-20 over the bundled sample documents. Figures are measured, not copied from published benchmarks. Reproduce them with the commands under "How to reproduce these measurements".

1. OCR model choice, trade-offs, and Bangla accuracy

Choice: Surya OCR (bn, en)

Surya was chosen because OCR accuracy is the first link in the chain — every downstream embedding and retrieval result is bounded by how faithfully the text was extracted. For Bangla specifically, the hard cases are **conjunct consonants** (যুক্তাক্ষর), **matra/vowel-sign placement**, and **dense multi-column layouts** (e.g. the bangla newspaper .jpg sample). Surya is a modern detection+recognition model with explicit layout handling and strong multilingual coverage, and it degrades more gracefully on these cases than a classical OCR pipeline.

The honest cost of that choice: Surya pulls in PyTorch and downloads model weights (~2–3 GB) on first run, and on a CPU-only machine it is slow (seconds per page). A GPU speeds this up significantly. A VLM-based OCR could push accuracy higher still, but at disproportionate setup and latency cost for this assessment's scope — hence Surya as the sweet spot.

Measured baseline

Measured on two contrasting Bangla+English inputs: `bilingual_sample.pdf` (a clean, **digitally-rendered** invoice, so it isolates OCR model quality from scan noise) and `bangla_invoice.png` (a **low-resolution, decoratively-styled** fruit-shop invoice — a deliberately hard real-world scan). Ground truth for the former is `data/sample_docs/bilingual_sample.html`.

Metric	Surya — clean digital PDF	Surya — low-res decorative scan
Char error rate, hand-checked paragraph	0.000 (Bangla <i>and</i> English Terms paragraphs verbatim)	high on the stylised title; body table mostly legible
Conjuncts (যুক্তাক্ষর) correct?	Yes — প্রযোজ্য, ক্ষেত্রে, মিনিকেট, পণ্য, সর্বমোট all correct	mostly yes in the item list (আঙ্গুর, পেঁপে, ড্রাগন)
Numerals ০-৯ correct?	Yes — ৫,৩১০ / ১৭০ / ১৩০ / ১৪৫ / ২,০০০ correct; one table cell ৮০ misread as "Ao"	mostly yes (১,২৩০ / ১,১৮০ / ১৪০) with occasional ০↔0 digit mixing (e.g. ২৮০.00)
Word-boundary / layout errors	table reading order was scrambled by detection order —	decorative header lost; table rows recovered

Metric	Surya — clean digital PDF	Surya — low-res decorative scan
	fixed by bbox reconstruction (see §2)	
Throughput (CPU-only)	34–57 s/page steady-state (1.1–1.8 pages/min); first page adds a one-time ~3–4 min model load	same

Honest read of the results. On clean digital Bangla, Surya is essentially character-perfect, including the conjuncts and matra placement that defeat classical OCR — the Terms paragraph came back at CER 0.000 in both scripts. Two real limitations showed up: (1) an **isolated numeric cell** (the unit price ৳০) was misread as Latin "Ao" — single short numeric tokens have little context to disambiguate; and (2) on the **degraded decorative scan**, the stylised shop title was unreadable and a few Bangla ০ digits came through as Latin 0. These are bounded by source-image quality, not by the pipeline — OCR is the first link and everything downstream inherits its ceiling. Throughput is the expected CPU cost of a transformer OCR model; a GPU removes it.

2. Chunking strategy and embedding-model selection

Why a multilingual embedding model is mandatory

A monolingual English embedding model maps Bangla text to a region of vector space that is geometrically unrelated to its English meaning. So a Bangla query against an English chunk — or vice versa — would score near-zero cosine similarity even when the two say the same thing. **BGE-M3** projects Bangla and English into a *single shared space*, which is what makes cross-lingual retrieval (English question → Bangla document) actually work on a bilingual corpus.

We use only BGE-M3's **dense** output (it also offers sparse and multi-vector modes). That is a deliberate scope choice: dense vectors + ChromaDB cosine search are sufficient for the hybrid-filter requirement, and skipping the extra modes keeps the pipeline simple.

Chunking: size, overlap, and the Bangla separator

- **Target $\approx$ 320 words/chunk with $\approx$ 50-word overlap.** This is a *retrieval-granularity* decision, **not** a token-limit workaround: BGE-M3 handles sequences up to 8192 tokens, so nothing is being truncated. Chunks too large dilute similarity and return imprecise context; too small and a chunk loses the context needed to answer. The overlap preserves meaning across chunk boundaries so an answer split across two chunks is still retrievable.
- **The splitter must treat the Bangla danda । (U+0964) as a sentence boundary** (plus II, then . ? ! ; , newlines). A recursive splitter that only knows Latin punctuation runs straight through Bangla sentences, cutting mid-thought and mangling context. This single addition is the difference between clean and broken Bangla chunks.

Tables: reading-order reconstruction (invoices)

The sample set is invoice-heavy, and tables are where naive OCR → chunk pipelines quietly fail. Surya returns detected text lines in **model/detection order**, which on a table interleaves a row's cells and detaches them from their column headers. On `bilingual_sample.pdf` the raw output looked like চাল (মিনিকেট) / Rice → ২৫ কেজি → ০১ → ২,০০০ → ৮০ jumbled across lines — the item, its quantity, and its price were no longer adjacent. A chunk built from that text still *contains* the numbers, but the **association** between a line item and its price is lost — exactly what a "what is the unit price of X?" query needs.

The fix lives in OCR flattening (`backend/ocr.py: :_surya_page_text`): each detected line carries a bounding box, so we **group lines into visual rows by vertical overlap and sort each row left-to-right**, reconstructing human reading order. After the fix the same table reads as সয়াবিন তেল / Soybean Oil ৫ লিটার / 5 L ১৭০ ৮৫০ — item beside its price.

For ordinary single-column prose every line is its own row, so output is unchanged (no regression).

This pairs deliberately with the chunk-size choice: a one-page invoice (~150 words) falls **entirely into a single chunk**, so the whole reconstructed table — headers, every line item, and the grand total — is retrieved as one coherent unit. That is why "What is the grand total?" → **5,310 BDT** and "unit price of soybean oil?" → ১৭০ both answer correctly. (Reading-order reconstruction can't repair a mis-recognised glyph: the one cell Surya misread, ৮০→"Ao", stays wrong — an OCR ceiling, not a chunking one.)

Per-chunk language detection feeds the filter

Language is detected **per chunk**, not per document, via a Unicode-range heuristic over letters only (Bangla block U+0980–U+09FF vs Latin a–z): `bn` if Bangla letters dominate ($\geq 85\%$), `en` if they're $\leq 15\%$, else `mixed`. A single bilingual document therefore yields a mix of `bn/en/mixed` chunks. That per-chunk language tag is exactly what makes the language filter meaningful at query time — filtering `language=bn` returns only the genuinely-Bangla chunks, not whole documents coarsely labelled.

Measured cross-lingual cosine (BGE-M3, this machine). Related Bangla ↔ English pairs score far higher than an unrelated pair, confirming the shared space works:

Pair	Cosine
বাংলাদেশের রাজধানী ঢাকা। ↔ "The capital of Bangladesh is Dhaka."	0.7536
আমি ভাত খাই। □ "I eat rice."	0.7865
বাংলাদেশের রাজধানী ঢাকা। ↔ "Quarterly revenue grew by twelve percent." (unrelated)	0.3176

The ~0.75–0.79 vs ~0.32 gap is exactly what makes an English query retrieve a Bangla chunk (and vice-versa). End-to-end this showed up as the English question "What is the unit price of soybean oil?" correctly returning ১৭০ from the Bangla/English invoice.

3. System architecture: metadata filtering + vector similarity

Full query data-flow

```
query text
  | (local) BGE-M3 embed -> query vector
  ▼
ChromaDB.query(query_embeddings=[v], where=<built from manual filters>,
n_results=k)
  | STEP A Boolean PRE-FILTER: keep only chunks whose metadata matches
  | `where`
  | STEP B cosine similarity ranking over ONLY the surviving chunks
  ▼
top-k chunks (+ metadata, distances)
  ▼
generate_answer(query, chunk_text, LLM_MODE, language_filter) # local (Ollama)
| none
  ▼
grounded answer (language follows the filter) + source citations
(filename, page, language, date, distance)
```

Why this is "hybrid"

Two mechanisms with different natures are combined:

- **Hard Boolean filter (metadata).** Built dynamically in `build_where()`. A single condition is a flat dict; multiple conditions are combined with an explicit `$and` (required by current ChromaDB). Empty filters are omitted — `where={}` is never sent.

Language filter is inclusive of mixed. Chunks are tagged `bn`, `en`, or `mixed` per the Unicode-range heuristic (§2). A bilingual invoice has `mixed` chunks — filtering strictly `{"language": {"$eq": "bn"}}` would exclude them, which is counter-intuitive (the user wants "chunks that contain Bangla"). So `language=bn` becomes `{"language": {"$in": ["bn", "mixed"]}}`, and likewise for `en`. Only `language=mixed` uses strict `$eq`.

Date ranges use a numeric `document_date_int` (YYYYMMDD), not the `date string`. ChromaDB 0.5.x rejects string operands for the range operators (`$gte/$lte` require `int/float` — passing `"2025-01-01"` raises `ValueError: Expected operand value to be an int or a float`). So at ingest we store the date both as a display string (`document_date`) and as a zero-padded integer (`document_date_int`, e.g. `20250314`) whose numeric order is chronological. Documents without a date **omit** `document_date_int`, so they are naturally excluded from any date-range filter.

```
def build_where(filters):
    conditions = []
    if filters.get("document_type"):
        conditions.append({"document_type": {"$eq":
filters["document_type"]}})
    lang = filters.get("language")
    if lang:
        if lang == "mixed":
            conditions.append({"language": {"$eq": "mixed"}})
        else:
```

```

        conditions.append({"language": {"$in": [lang, "mixed"]}})
        date_from = date_to_int(filters.get("date_from")) # "2025-01-01" ->
20250101
        date_to = date_to_int(filters.get("date_to"))
        if date_from is not None:
            conditions.append({"document_date_int": {"$gte": date_from}})
        if date_to is not None:
            conditions.append({"document_date_int": {"$lte": date_to}})
        if not conditions:
            return None
        return conditions[0] if len(conditions) == 1 else {"$and": conditions}

```

- **Soft semantic ranking (vectors).** Cosine similarity over the *filtered* candidate set only.

This is **filter-then-search**: the manual filters are a hard constraint that shrinks the candidate pool *before* similarity is computed — not a re-ranking nudge applied afterward. Neither mechanism alone is sufficient: pure semantic search can't honour "only invoices dated 2024"; pure metadata filtering can't rank by meaning.

Verified end-to-end (this run)

Three invoices ingested — `bilingual_sample.pdf` (mixed, 2025-03-14), `bangla invoice.png` (bn, 2024-05-20), `english invoice.png` (en, 2023-10-27) — then the **same query** "invoice total amount" under different filters:

Filter	Chunks returned	Note
(none)	all 3 (bilingual ranked top)	pure semantic
language=bn	bangla only → answer 1,180.00	hard language pre-filter
language=en	english only → answer \$3,966.30	hard language pre-filter
date_from=2025-01-01, date_to=2025-12-31	bilingual only	numeric date range
date_from=2024-01-01 (open upper)	bangla + bilingual (≥2024)	open-ended range
language=bn + 2025 range	0 chunks → "I don't know" + warning	the bn doc is 2024 → composed \$and excludes it
document_type=report	0 chunks → graceful warning	no such type ingested

The `language=bn + 2025-range` case is the crisp proof that the two filter dimensions compose as a conjunction *and* that the date bound is genuinely applied: semantically the Bangla invoice is the best match, but the date filter correctly removes it, and the system says "I don't know" rather than fabricating.

Grounded, language-aware answer generation (the final step)

Once the top-k chunks are retrieved, `generate_answer()` (backend/`rag.py`) composes the reply with the local Ollama model under a strict contract designed for accuracy:

- **Strict grounding.** The system prompt forces the model to use *only* the retrieved context —

no outside knowledge, no inference, no arithmetic. If the answer is not literally present it must reply exactly *"I don't know based on the provided documents."* This is why the empty-filter and absent-fact cases refuse instead of fabricating.

- **Deterministic decoding.** Greedy settings (temperature 0, top_k 1, top_p 1, fixed seed) mean the same context + question always yields the same answer and the model can't "drift" into invented detail — the main lever against hallucination on a small local model.
- **Language follows the filter.** The selected language filter also drives the answer language: bn → Bangla, en → English, mixed/any → match the question. A user who filters to Bangla content wants a Bangla answer, regardless of the question's language.
- **Verbatim values + currency normalisation.** Numbers and dates are copied digit-for-digit. The one display-level normalisation is currency: the invoices abbreviate Taka as (ট) in table headers, so a deterministic post-step expands that to the full word — টাকা in a Bangla answer, BDT in English — without touching the number. Hence *"unit price of soybean oil?"* → ১৭০ টাকা (bn) / ১৭০ BDT (en), and *"grand total?"* → ৫,৩১০ টাকা / 5,310 BDT.

The answer is always shown alongside its **source citations** (filename, page, per-chunk language, type, date, cosine distance), so every claim is verifiable against the exact chunk it came from.

Locality — fully local, no exceptions

The brief requires processing "without sending data to external commercial APIs." This system makes **no external API calls at any stage**:

- **OCR (Surya) and embedding (BGE-M3) run 100% on-machine** — no network call ever touches a raw file, image, or full OCR text.
- **Answer generation runs locally too.** The default LLM_MODE=local performs synthesis with a local Ollama model (qwen2.5:7b) over localhost, so the entire pipeline — OCR → embedding → retrieval → synthesis — is on-machine with zero external calls.
- LLM_MODE=none is a dependency-free alternative for anyone without Ollama: it returns the retrieved, cited chunks verbatim (also fully local).

The only network traffic in local mode is a localhost:11434 request to the Ollama daemon on the same machine — nothing leaves the host. So the honest one-line answer to "is it local?": yes, *entirely* — OCR, embedding, retrieval, and answer generation all run on-device with no external/commercial API.

Under-filled / empty result edge case

If the filter combination is too narrow, /search returns whatever matched and attaches a warning:

- 0 matches → "No chunks matched ... try relaxing the filters."
- fewer than top_k → "Only N chunk(s) matched (requested top_k=...)."

The system never silently fabricates an answer from a near-empty context.

How to reproduce these measurements

```
# 1. OCR baseline (prints raw text + timing per page)
python -m scripts.ocr_baseline "data/sample_docs/bilingual_sample.pdf"
python -m scripts.ocr_baseline "data/sample_docs/bangla_invoice.png"

# 2. Cross-lingual embedding sanity check (the cosine table in §2)
python -m scripts.embed_check

# 3. End-to-end via the API (see README/GUIDELINE) and read the /search
#    distances + the per-source language/date metadata
```

The numbers above were produced by exactly these commands on the hardware noted at the top of this document; re-running reproduces them (CPU timings vary by machine).